



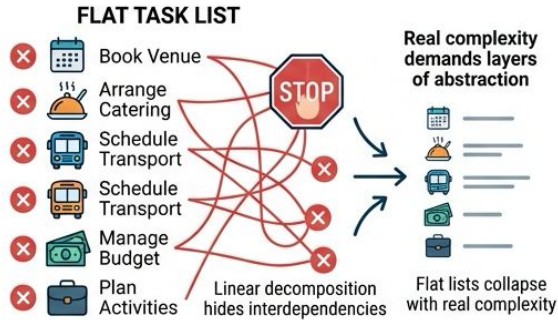
U.S. General Services
Administration

Chapter 5.4: Hierarchical Planning Fundamentals

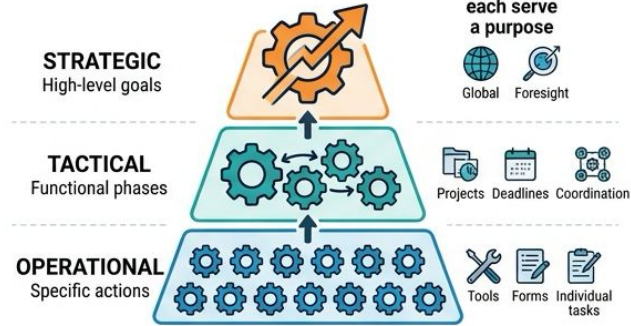
US General Service Administration
AI Community of Practice - March 2026

The Hierarchical Tree

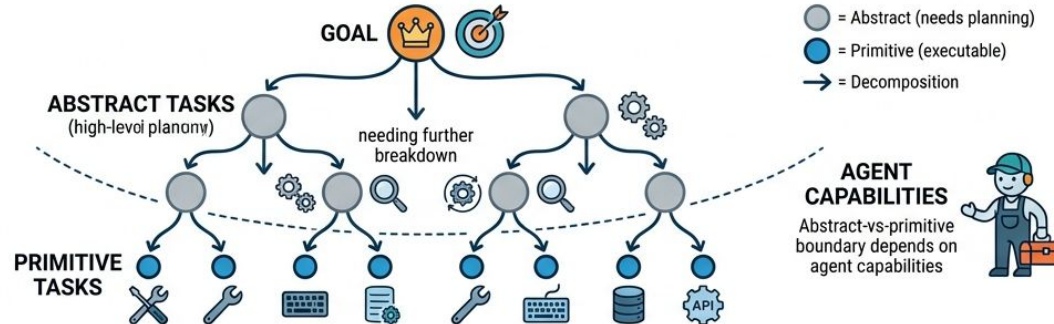
When Flat Task Lists Hit a Wall



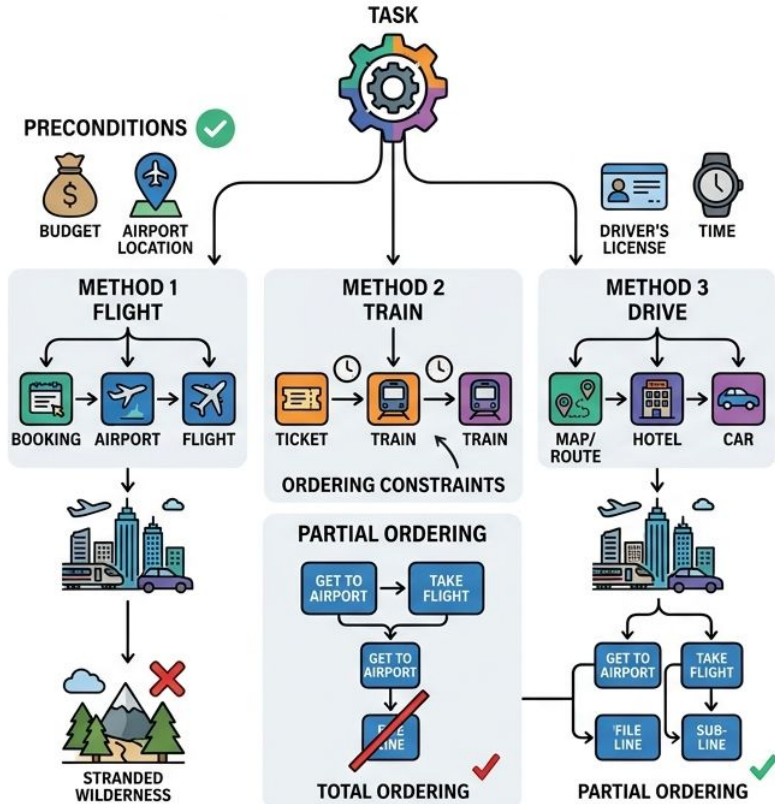
Hierarchical Planning Scales



The Hierarchical Tree: Goals, Tasks, and Primitives

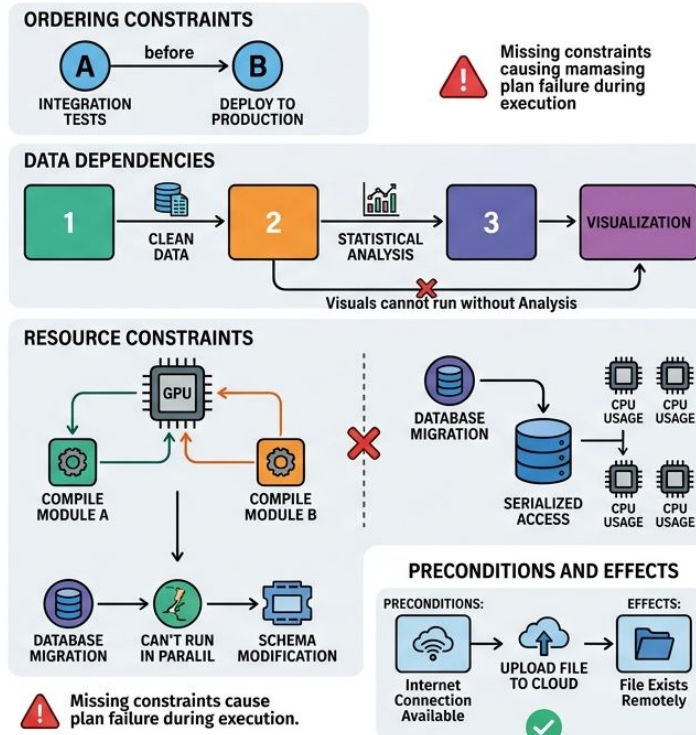


Decomposition Methods -- The Secret Sauce



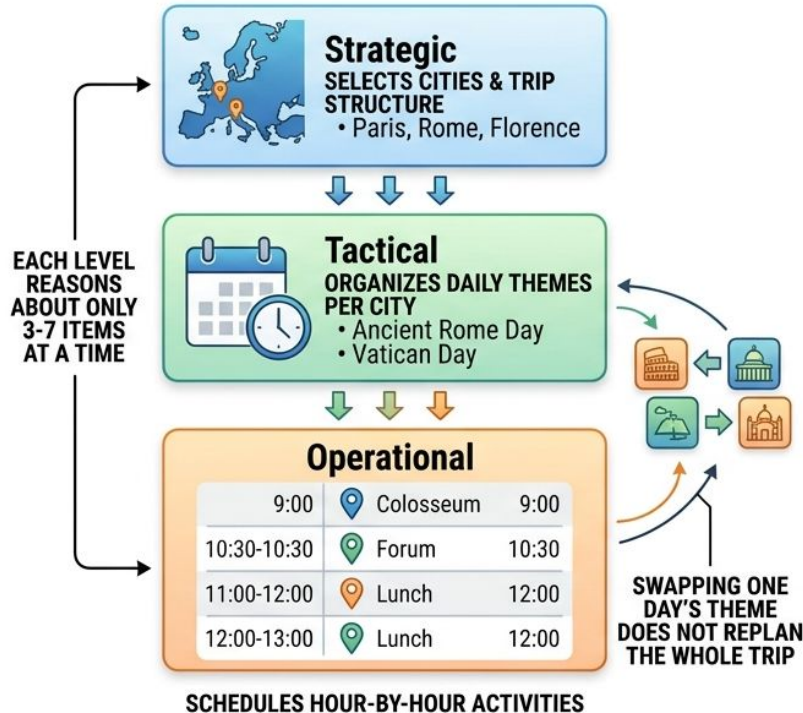
- Methods map abstract tasks to specific subtask sequences
- Same task can have multiple valid methods for different contexts
- Each method carries preconditions gating its applicability
- Ordering constraints specify which subtasks can run in parallel
- Partial ordering maximizes execution flexibility

Task Dependencies and Constraints



- Ordering constraints enforce "task A before task B"
- Data dependencies ensure outputs flow to consuming tasks
- Resource constraints prevent conflicts over shared assets
- Preconditions and effects model state changes formally
- Missing constraints produce plans that fail at execution time

Case: The European Vacation

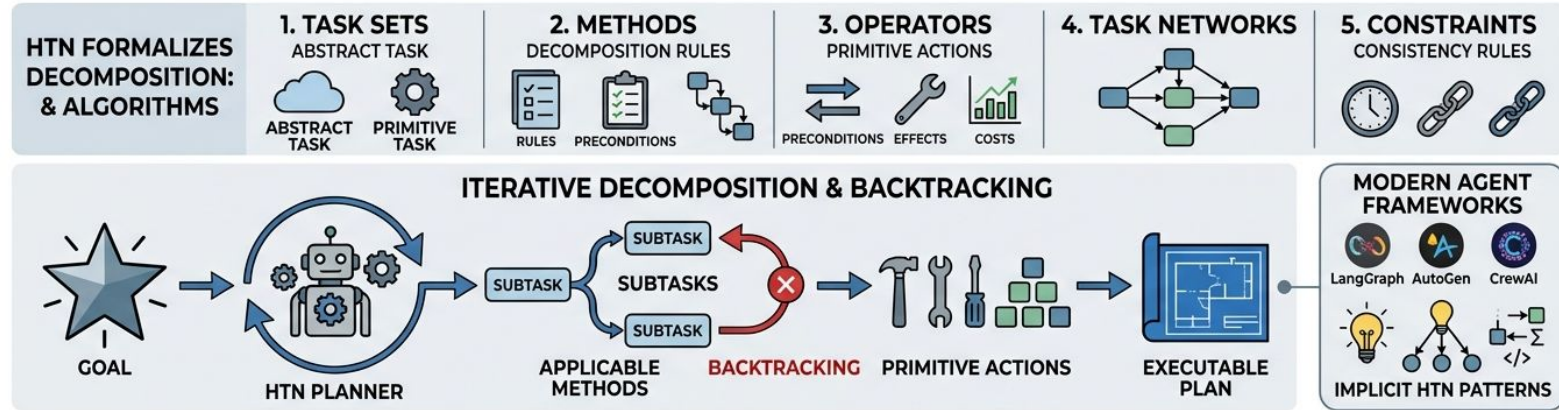


- Strategic level selects cities and trip structure
- Tactical level organizes daily themes per city
- Operational level schedules hour-by-hour activities
- Each level reasons about only 3-7 items at a time
- Swapping one day's theme does not replan the whole trip

Why Hierarchical Beats Flat

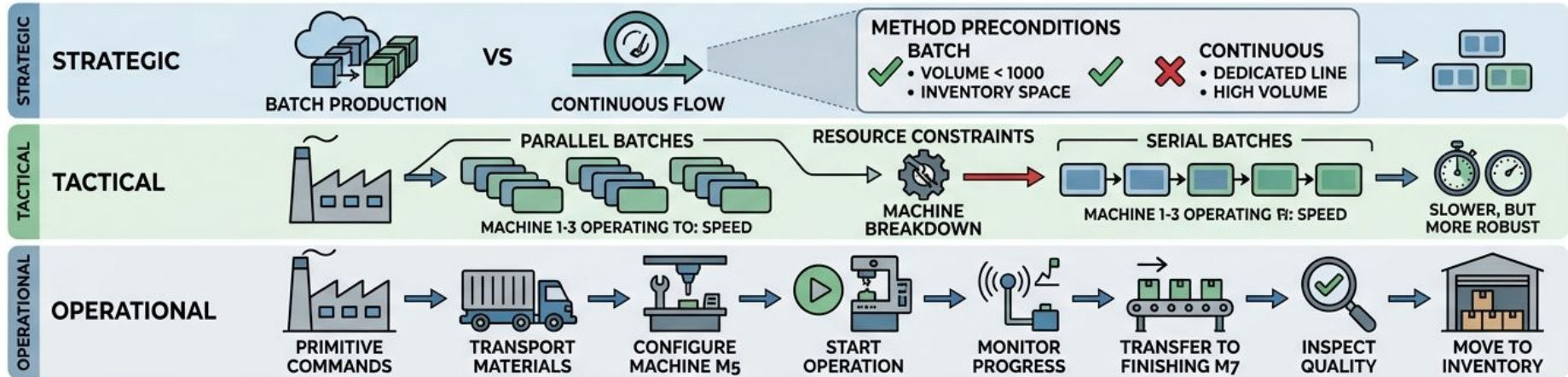
- Flat planning searches factorial orderings of all actions
 - Adding one action exponentially increases search space
 - Hierarchical planning decomposes search into small decision sets
 - Transforms exponential complexity into polynomial complexity
 - Tradeoff: may miss globally optimal cross-cutting solutions
- 

HTN Planning -- The Formal Framework

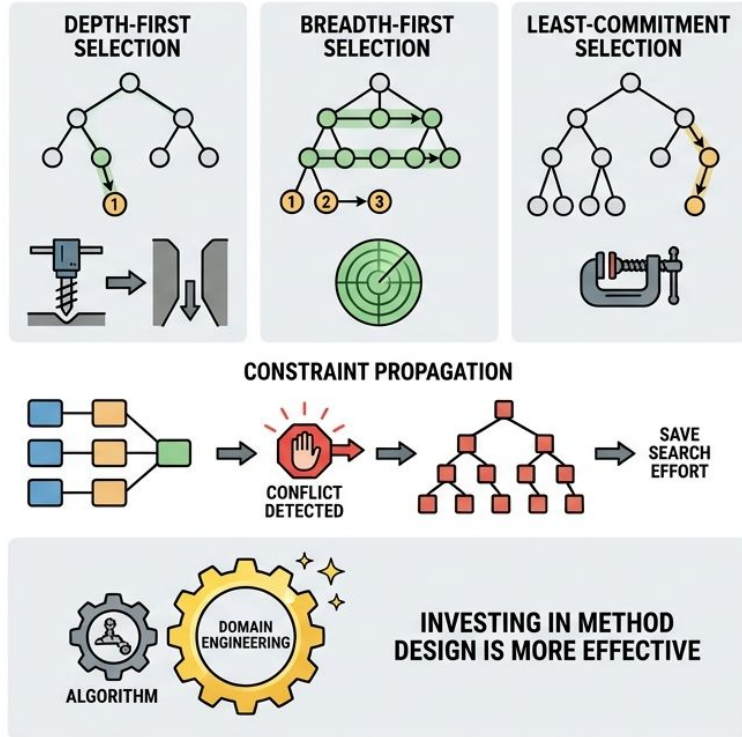


- Hierarchical Task Networks formalize decomposition as algorithms
- Five components: task sets, methods, operators, networks, constraints
- HTN planner iteratively decomposes until only primitives remain
- Backtracking explores alternative methods when paths fail
- Modern agent frameworks implicitly implement HTN patterns

Manufacturing Example



Task Selection and Search Strategies

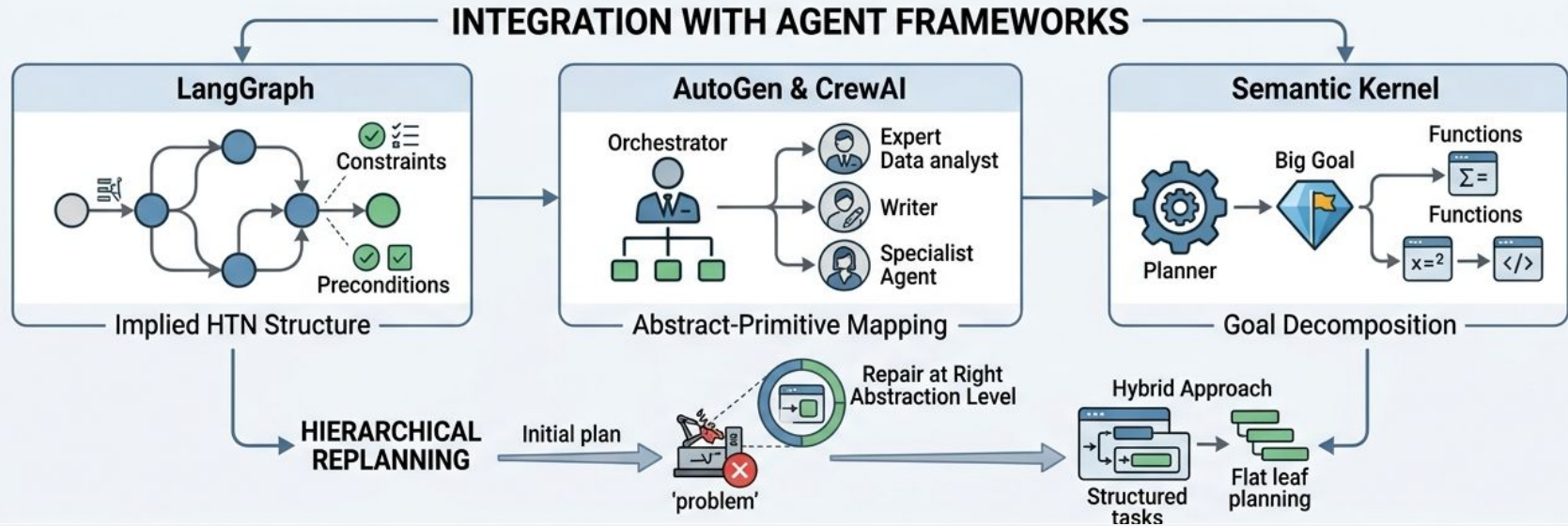


- Depth-first decomposes one branch completely before siblings
- Breadth-first expands all tasks at one level before going deeper
- Least-commitment tackles most constrained tasks first
- Constraint propagation detects failures early, saving search effort
- Domain engineering matters more than algorithm sophistication

Common Pitfalls and Misconceptions

- Abstraction levels reflect decision relevance, not time scales
 - More hierarchy depth does not always improve performance
 - Ignoring shared resources produces plans that fail at execution
 - Method libraries need structure, not just quantity
 - Primitive tasks can fail -- always plan for execution monitoring
- 

Integration with Agent Frameworks



Key Takeaways

- Hierarchical planning transforms exponential search into manageable layers
 - HTNs formalize decomposition with methods, constraints, and operators
 - Use HTN when tasks are nested, reusable, and multi-stakeholder
 - Skip HTN for simple sequences or highly dynamic environments
 - Next up: Monte Carlo Tree Search for probabilistic planning
- 